Math 3680: Week 12

Arjan Singh Ellingson

California Polytechnic State University

November 9, 2026

Introduction

GraphQL (you may see gql) may is an API method that allows for communication between a client or user and a server.
GraphQL is an alternative to the more common `REST` format used in web development.
Calls are executed from files with the extension `.gql` or `.graphql`.
It is not limited to a specific database or backend language.

Brief History & Alternatives

1. started by Facebook in 2012, aiming to becoming open source by 2015

[1] https://en.wikipedia.org/wiki/GraphQL

Brief History & Alternatives

1. started by Facebook in 2012, aiming to becoming open source by 2015
2. 2018: GraphQL was adopted by GraphQL Foundation, with parent Linux Foundation[1]

[1] https://en.wikipedia.org/wiki/GraphQL

Brief History & Alternatives

1. started by Facebook in 2012, aiming to becoming open source by 2015
2. 2018: GraphQL was adopted by GraphQL Foundation, with parent Linux Foundation[1]
3. Currently: APIs of companies like Facebook, GitHub (`gh cli`), Yelp, and Shopify utilize gql format

[1] https://en.wikipedia.org/wiki/GraphQL

Brief History & Alternatives

1. started by Facebook in 2012, aiming to becoming open source by 2015
2. 2018: GraphQL was adopted by GraphQL Foundation, with parent Linux Foundation[1]
3. Currently: APIs of companies like Facebook, GitHub (`gh cli`), Yelp, and Shopify utilize gql format
4. GraphQL combines each of REST's endpoints into one endpoint (usually called `/graphql`)

[1] https://en.wikipedia.org/wiki/GraphQL

Brief History & Alternatives

1. started by Facebook in 2012, aiming to becoming open source by 2015
2. 2018: GraphQL was adopted by GraphQL Foundation, with parent Linux Foundation[1]
3. Currently: APIs of companies like Facebook, GitHub (`gh cli`), Yelp, and Shopify utilize gql format
4. GraphQL combines each of REST's endpoints into one endpoint (usually called `/graphql`)
5. The Problem: Rest often **overfetches** or **underfetches** the data required.

[1] https://en.wikipedia.org/wiki/GraphQL

Structure of a GraphQL Call

A basic call is formatted as

```
schema {
    query: Query
    mutation: Mutation
}
type Query {}
type Mutation {}
```

Example from a different Coding Project

Example from a different Coding Project

```
type ArtistNode
  id: ID!
  label: String!
  genre: String
  era: String
  image: String
  kind: NodeKind
  visible: Boolean!
```

Example from a different Coding Project

```
type ArtistNode
  id: ID!
  label: String!
  genre: String
  era: String
  image: String
  kind: NodeKind
  visible: Boolean!


type RelationshipEdge
  id: ID!
  source: ArtistNode!
  target: ArtistNode!
  type: String!
  directed: Boolean!
  year: Int
  note: String
  song: SongRef
```

Example from a different Coding Project

```
type ArtistNode
  id: ID!
  label: String!
  genre: String
  era: String
  image: String
  kind: NodeKind
  visible: Boolean!
```

```
type RelationshipEdge
  id: ID!
  source: ArtistNode!
  target: ArtistNode!
  type: String!
  directed: Boolean!
  year: Int
  note: String
  song: SongRef
```

```
type Query
  artists(visibleOnly: Boolean = false): [ArtistNode!]!

  artist(id: ID!): ArtistNode

  edges(type: String, targetLabel: String): [RelationshipEdge!]!
```